

These VNFs are software versions of network functions like load balancers, firewalls, and VPNs that can run on standard hardware instead of specialised networking equipment. The VNF descriptor includes information about the VNF's capabilities, such as input and output parameters, supported protocols, and other relevant details. Typically, the VNF descriptor is defined in a standard format like OpenStack TOSCA, making it easy for VNF management tools to process and understand. It is important to note that TOSCA is a standard developed by OASIS (Organisation for the Advancement of Structured Information Standards) that provides a language for describing cloud services and their relationships, as well as an API for managing those services.

In NFV systems, VNF descriptors automate the deployment and maintenance of VNFs. They allow administrators to declaratively specify the characteristics and behaviours of VNFs, enabling the automation of resource provisioning and VNF configuration. The sample VNFD file is seen in Figure 1.

The VNFD (virtual network function descriptor) provides a detailed description of VNFs, which are categorised into three basic node types: virtual deployment unit (VDU), connection point (CP), and virtual links (VL).

A VDU specifies the virtual machine (VM) instance that will host the VNF, including its monitoring policies, the image that will be used, and the flavour. It is similar to Nova servers in OpenStack.

The CP represents the virtual network interface card (NIC), which is virtually connected to a VDU. It is also known as single root I/O virtualisation (SR-IOV) NIC. This node type provides information about the logical virtual link object that

```
tosca_definitions_version: tosca_simple_p
                                _profile_for_nfv_1_0_0
description: OpenWRT with services
metadata:
  template_name: OpenWRT
topology_template:
  node_templates:

    VDU1:
      type: tosca.nodes.nfv.VDU.Tacker
      capabilities:
        nfv_compute:
          properties:
            num_cpus: 1
            mem_size: 512 MB
            disk_size: 1 GB
      properties:
        image: OpenWRT
        config: |
          param0: key1
          param1: key2
        mgmt_driver: openwrt
        monitoring_policy:
          name: ping
          parameters:
            count: 3
            interval: 10
          actions:
            failure: respawn

    CP1:
      type: tosca.nodes.nfv.CP.Tacker
      properties:
        management: true
        order: 0
        anti_spoofing_protection: false
      requirements:
        - virtualLink:
            node: VL1
        - virtualBinding:
            node: VDU1
```

Figure 1: A sample VNFD file

connects VDUs, and it corresponds to Neutron ports in OpenStack.

Lastly, the VL node type provides details about the logical virtual link object that connects VDUs, like Nova networks in OpenStack. It describes the connections between VDUs and can be used to specify network policies, such as bandwidth or quality of service (QoS) requirements. The VNFD's standard format, such as the OpenStack TOSCA format, enables administrators to easily automate the deployment and configuration of VNFs in NFV environments.

Steps for VNF deployment in OpenStack

Step 1: To ensure proper organisation of Tosca templates, it is recommended to create a separate folder named 'tacker' that will hold all the templates

related to Tacker. This will make it easier for administrators to locate and manage these templates. Figure 2 shows an example of how the folder structure may look.

Step 2: Launch OpenStack on the browser.

Step 3: We need to create a network topology to be able to attach VNF to the desired network which is present under the network tab.

Step 4: To instantiate a VNF, we first need to load the corresponding TOSCA template to the VNF catalogue. Figure 3 shows the VNF catalogue interface, where the 'Name' tab displays the list of VNF templates that have been uploaded. Each template is assigned a unique catalogue ID and can be given a user-defined description and service type. The 'Onboard VNF' button allows the user to upload a new VNF template to the catalogue.

Step 5: We need to create a virtual infrastructure for VNF deployment. For that, we need to create a `vim_conf.yaml` file with the properties shown in Figure 4.

Step 6: Once the `vim_conf.yaml` is ready, one needs to create a virtual infrastructure manager (VIM) with OpenStack using the following command:

```
$ openstack vim register --description
'OpenStack vim' -- config-file vim_config.
yaml Site1
```

This command will be used to register a VIM, which requires a configuration file. In our case, this is `vim_config.yaml` via the OpenStack command and the `--description` tag is used to describe the VIM that will be created in Site 1.

```
desktop [~/tacker] Documents Downloads Music Pictures Public snap tacker Templates Videos
astha@astha-VirtualBox:~$ cd tacker
astha@astha-VirtualBox:~/tacker$ ls
openwrt_sample.yaml sample-tosca-vnfd-flavour.yaml simple-vnf.yaml tosca-vnfd-multi-vm.yaml tosca-vnfd-scale.yaml
astha@astha-VirtualBox:~/tacker$
```

Figure 2: Tacker folder details